

7 Prediction

Viterbi

Given observations and a trained model, let's estimate the states.

Given observations in time and a hidden Markov model, determine the most likely state sequence.

The model Θ which maximizes the chance of getting the observations o from a sequence of states s is the "best" one.

At every time step, consider the total number of states that you could be in.

If there are n time steps and i states, we have i^n combinations to search. This is not tractable.

$$S^* = \operatorname{argmax} (P(o, s | \Theta))$$

We have

$$\begin{aligned} P(o, s) &= P(o_1, \dots, o_N) \\ &= P(S_1)P(o_1|S_1)P(S_2|PS_1, O_1)P(O_2|S_2, d_1, O_1)P(S_3|S_2, O_2S_1O_1) \\ &= P(S_1)P(O_1|S_1)P(S_2|S_1)P(O_2|S_2)P(S_3|S_2) \dots \\ &= P(S_1)P(O_1|S_1) \cdot \prod_{n=2}^N P(s_n|S_{n-1})P(O_n|S_n) \end{aligned}$$

$$\ln(P(O, S)) = \underbrace{\ln P(s_1) + \ln P(O_1|S_1)}_{\text{initialie}} + \underbrace{\sum_{n=2}^N (\ln P(s_n|s_{n-1}) + \ln P(o_n|s_n))}_{\text{transitions}}$$

We can rephrase this to finding the highest scoring path through a matrix.

We are now computing the pairwise similarity matrix and get the path with the highest score.

The similarity matrix is

For example, the multivariate gaussian

α_i	$P(o_1 \alpha)$				
α_2	$P(o_1 \alpha_2)$				
α_1	$P(o_1 \alpha_1)$	$P(o_2 \alpha_1)$			
	o_1	o_2	o_3	$\dots$	o_n

In this case, we can be in any state at any time. We are moving between each time step observation from left to right, but with the maximum probabilities.

The transition score is often modified to

$$\ln P(s_n|s_{n-1}) + \underbrace{\hspace{1cm}}_{\text{hyper parameter}} \cdot \ln P(o_n|s_n)$$

Learning Model

Given observations and states, how do we train a model?

Let's say we have a bunch of observations $O = (O_1, O_2, O_2, \dots)$, and the state sequence that it actually corresponds to.

We need the state transitions, and the parameters of a Gaussian. Each model is given by

$$\Theta = (\pi, A, B, \dots)$$

Where π is the state transition matrix and A is the probability of starting in a state.

The transitions are given as

$$a_{ij} = \frac{a_{ij}}{\sum_j a_{ij}}$$

The initial state probability is either

- manual
- averaged over all the states
- averaged over all first states

i

The average of all observations from the state α_i

This is still an n dimensional vector, its the means of all the states when we know that it is actually in state i .

i

$$\mu_i = \frac{1}{N_i - 1} \sum_{n=1}^{N_i} (x_i^n - \mu_i)(x_i^n - \mu_i)$$

This is computing an outer product with itself, since the dimensions are

$$1 \times 1$$

These are the observations that we've collected for a specific state.

We are taking all the data of a single state throughout training, and we find the covariance, and mean for that state.

Forced Alignment

From weak to strong classification

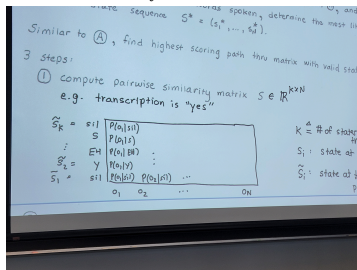
Given observations, a trained model, and a transcription of the words spoken, find the most likely state sequence.

We have weak labels - we need to label data to train models, but annotating on the level of frames is hard. If we can do it with a transcription, it's far easier for humans but also harder to compute.

Compute a similarity matrix, use dynamic programming to find the cumulative best path, and backtrace.

Lets say we know the transcription is yes. We make a starting state of silence, and ending state of silence, and we arrange the states in assending order - y, eh, s.

Lets define s_i as the state at audio frame i , and s_i be the state at transcription position i .



We allow transitions diagonally up and right, or just right - you can't have two states at one point in time.

We want to know which of those paths has the best score, so we compute the dtw.

(i, k) is the optimal cumulative path score up to the (i, j) element

We initialize with silence as the starting state

$$(i, k) = \begin{cases} \ln P(\text{sil}) + \ln P(o_1 | \text{sil}) & i = 1 \\ -\infty & i = 2, \dots, k \end{cases}$$

We then make the matrix dynamically

$$(\underbrace{i}_{\text{transcription state}}, \underbrace{j}_{\text{rame}}) = \begin{cases} (i, j-1) + \ln a_{s_i, s_i} + \ln P(o_j | s_i) \\ (i-1, j-1) + \ln a_{s_{i-1} s_i} + \ln P(o_j | s_i) \end{cases}$$